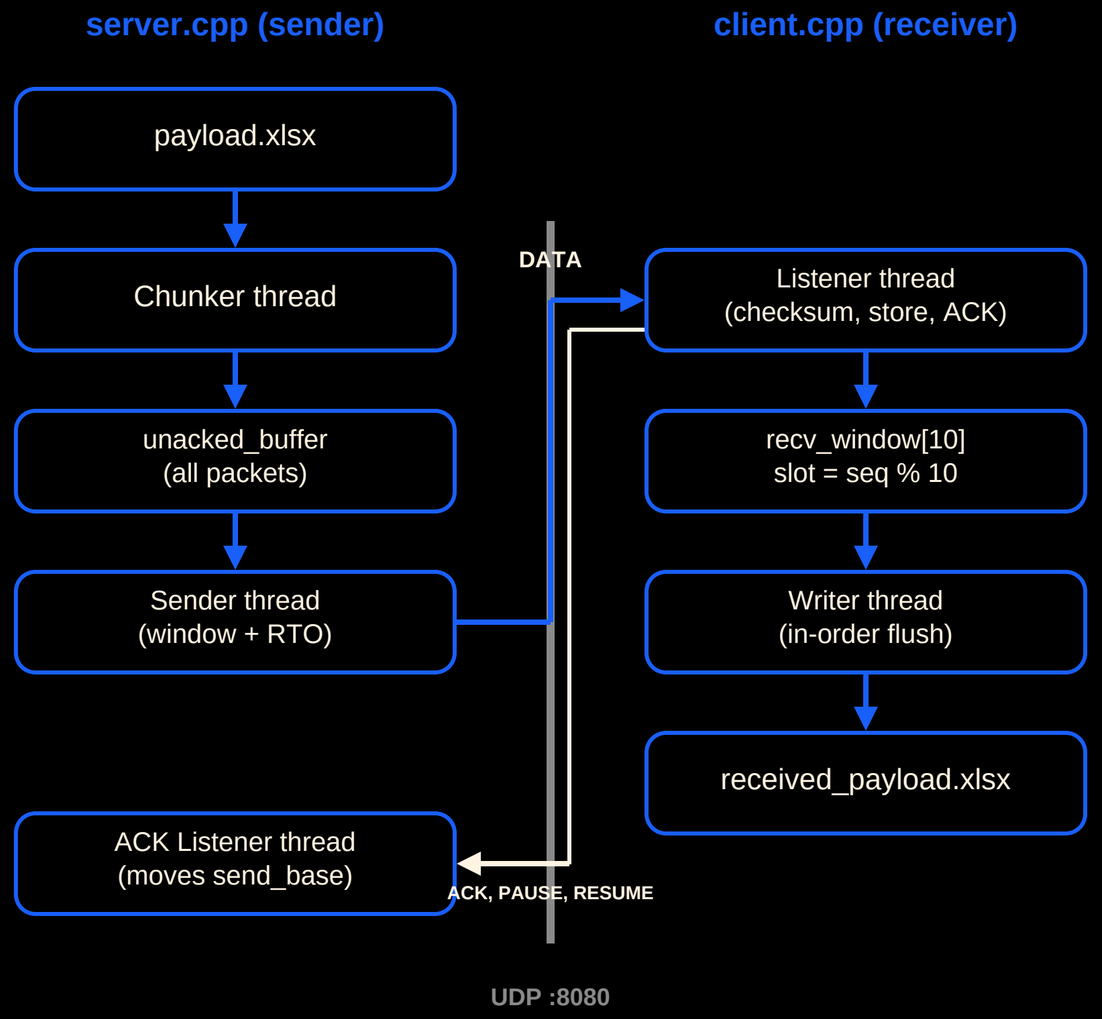
Reliable File Transfer over UDP

A TCP-style transport written in C++ on raw UDP sockets. server.cpp sends payload.xlsx from the server folder; client.cpp rebuilds it as received_payload.xlsx in the client folder.

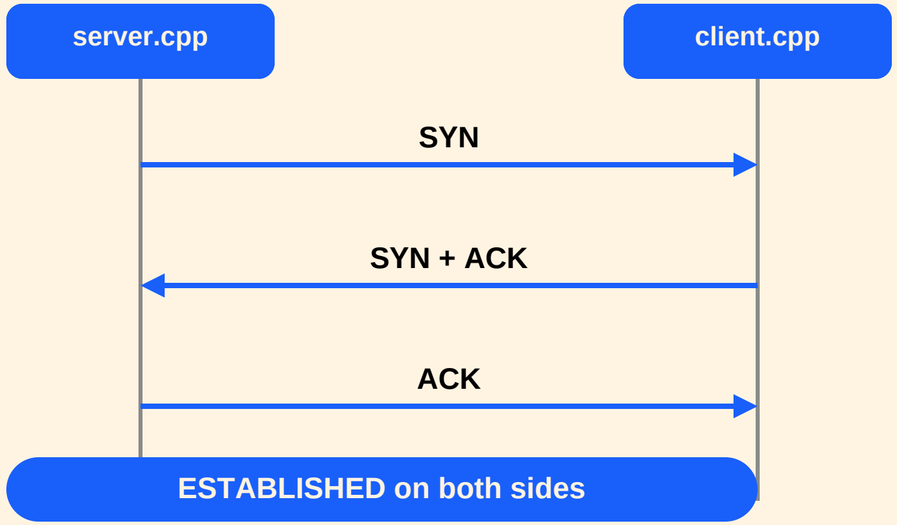
What is implemented

- 3-way handshake (SYN, SYN+ACK, ACK)
- Chunking into 1024-byte packets with checksum and EOF flag
- Sliding window of 10, cumulative ACKs
- RTO timer (value typed at start) and fast retransmit on 3 duplicate ACKs
- 3 modes: window advertising + zero-window probe, PAUSE/RESUME, drop every 3rd packet
- 4-way teardown with TIME_WAIT (2 x MSL)
- Event logs replayed in a browser visualiser



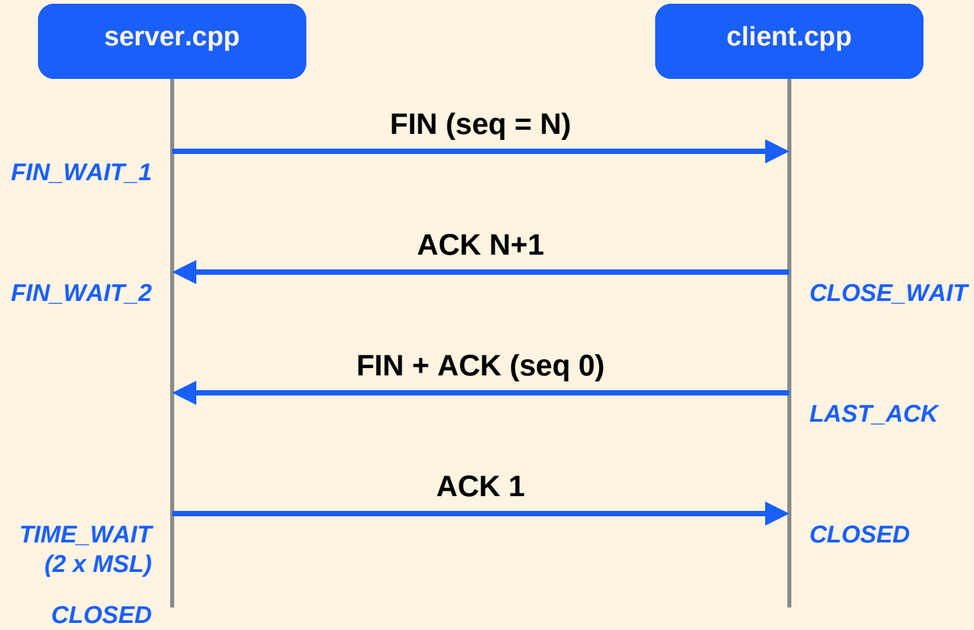
Connection setup and teardown

3-way handshake



- Receiver binds UDP 8080 and blocks until a SYN arrives
- Sender waits for SYN+ACK before starting its threads
- Then a 100 ms receive timeout (SO_RCVTIMEO) is set so every thread can notice the end of transfer and exit

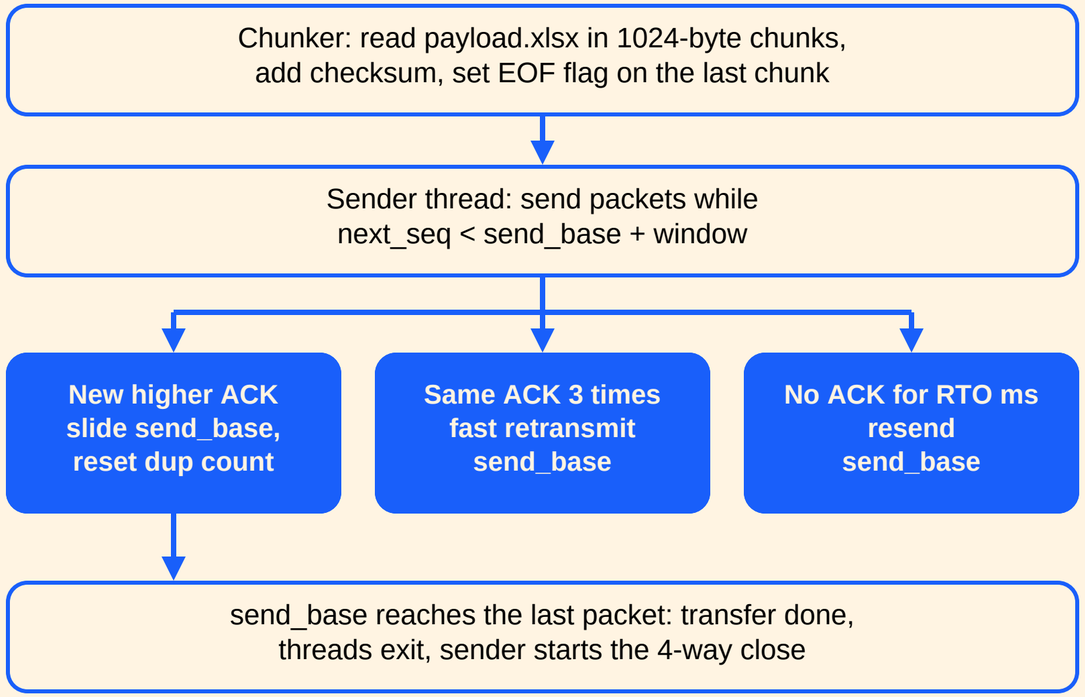
4-way termination



- FIN resent on RTO, gives up after 5 tries
- A FIN that arrives while data is still being flushed is remembered and handled after the writer finishes
- A repeated FIN during TIME_WAIT is ACKed again

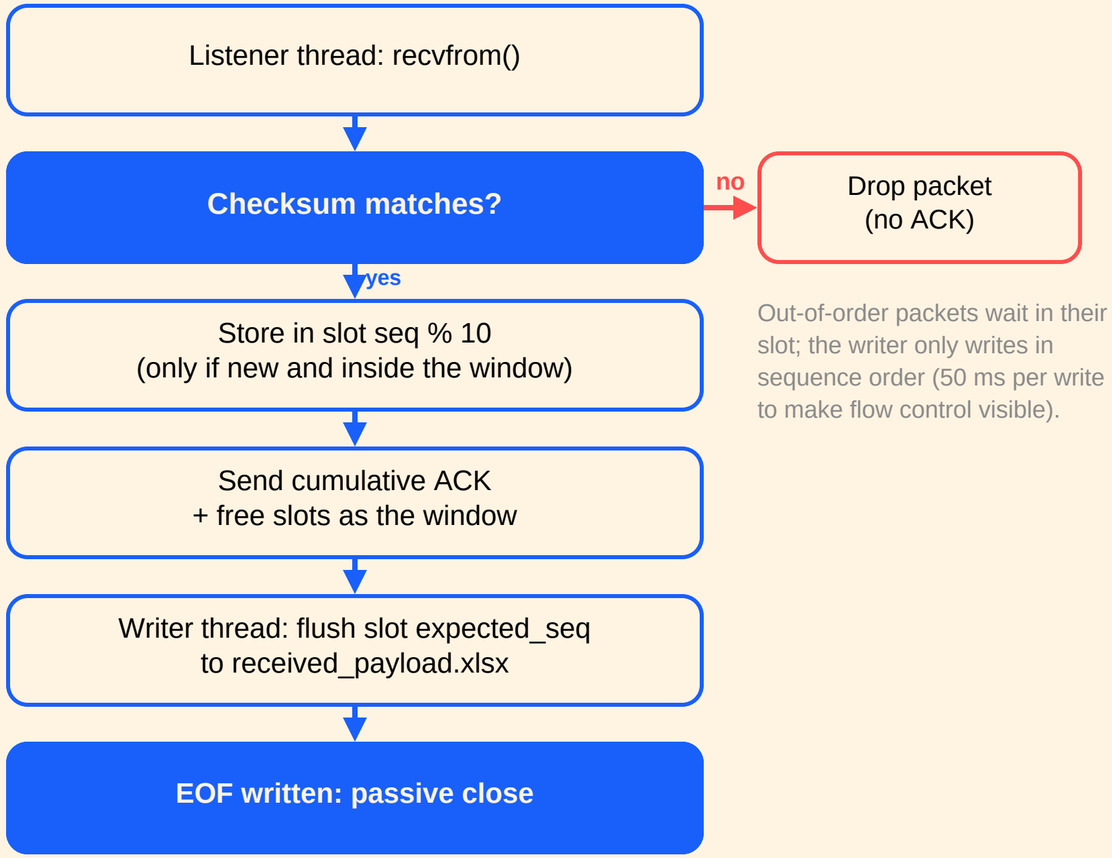
Reliable data transfer

Sender loop (server.cpp)



RTO value is typed in when the sender starts. The ACK Listener thread updates send_base and the advertised window under one mutex.

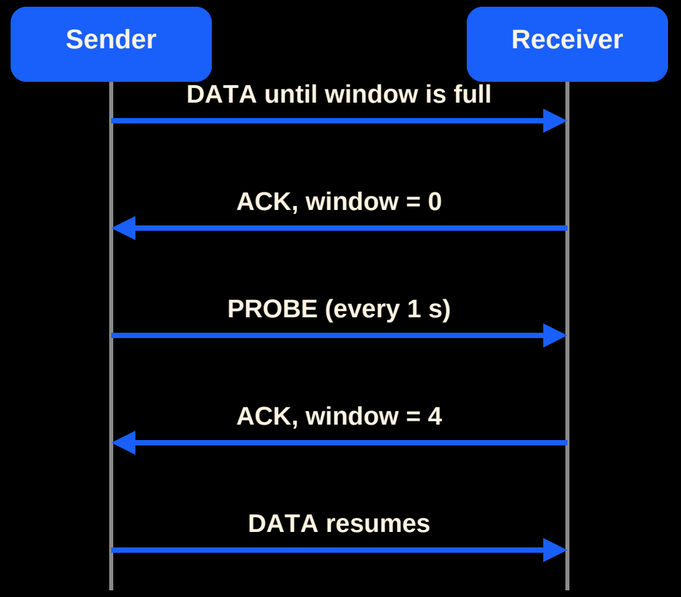
Receiver pipeline (client.cpp)



Out-of-order packets wait in their slot; the writer only writes in sequence order (50 ms per write to make flow control visible).

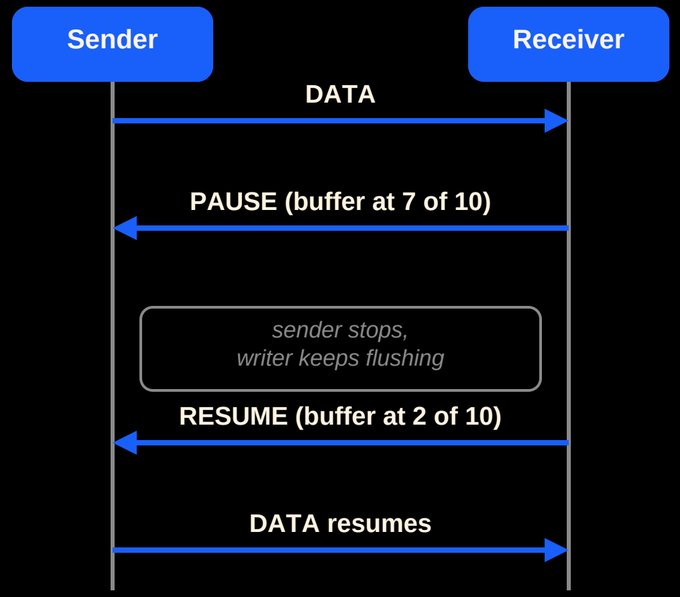
Three modes, chosen from a menu on both sides

Mode 1: Window advertising



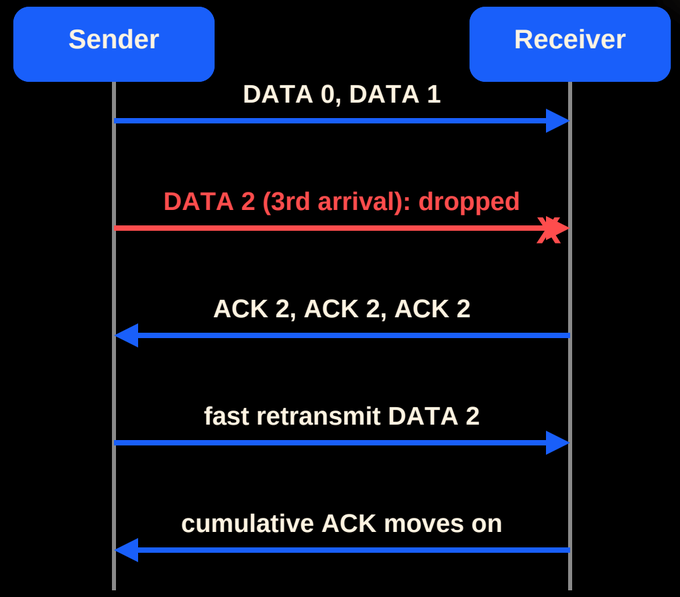
Each ACK carries window = 10 minus filled slots. The sender never has more than that in flight; at 0 it sends a PROBE every second until space opens.

Mode 2: PAUSE / RESUME



The receiver watches its own buffer and sends explicit PAUSE and RESUME flags. The sender stops pushing data while paused and uses a fixed window of 10.

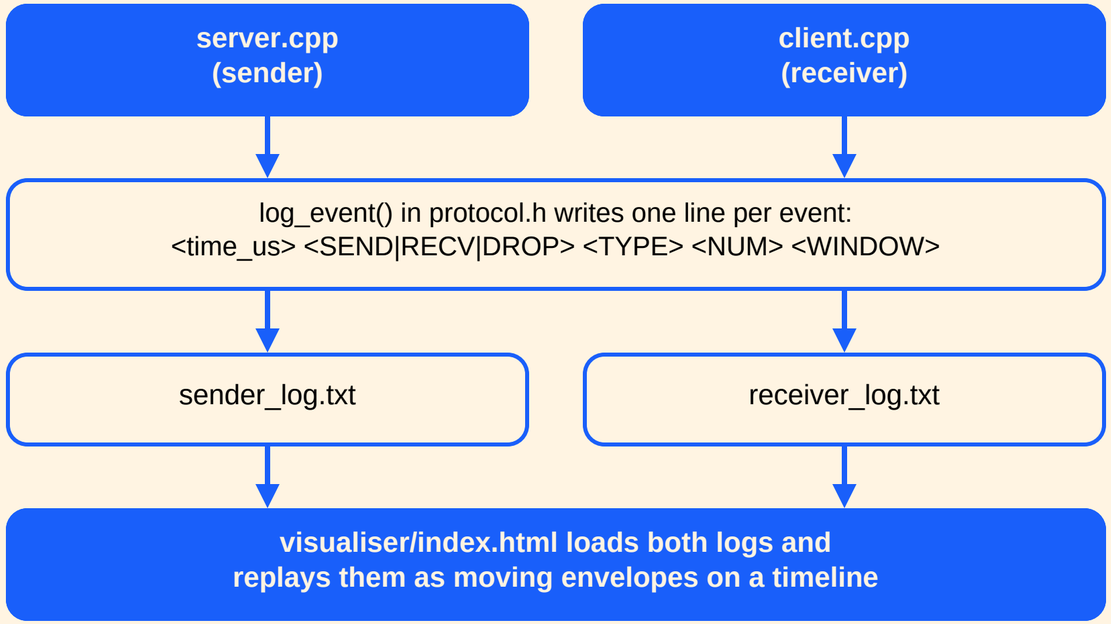
Mode 3: Drop every 3rd packet



The receiver counts every DATA arrival (retransmissions too) and discards each 3rd one before the checksum: not stored, not ACKed, logged as DROP. Flow control works as in mode 1.

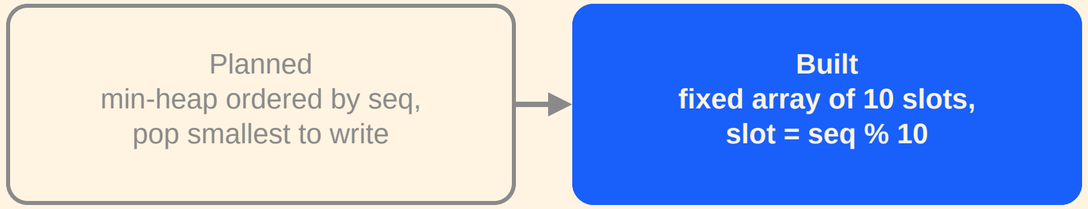
Logging, visualiser and the buffer design

Every packet event is logged and replayed



A SEND with no matching RECV on the other side is drawn as lost, so mode 3 drops and the retransmissions that follow are visible step by step. Plain HTML, CSS and JS, no libraries.

Planned vs built: receive buffer



Why the heap was dropped

- Flow control needs a fixed-size buffer: the advertised window is 10 minus filled slots, and PAUSE / RESUME fire at 7 and 2 of 10. A heap grows, so there is no natural “free space” to report.
- The window already caps how far ahead a packet can be, so seq % 10 never collides: direct O(1) placement and lookup.
- The writer needs exactly expected_seq; a heap only exposes its smallest item and still needs a separate duplicate check (received_seqs set).

How to run

g++ -std=c++17 -pthread on both files → start client first → pick the same mode (1, 2 or 3) on both sides → enter the RTO on the sender → received_payload.xlsx appears in client/